

AgentFlow: A Human-Agent Collaborative Visual Workflow Instrument via WebMCP

Parithosh Varma · AgentFlow Team · WebMCP Hackathon 2026

Human × Agent Canvas — <https://agentflow-hackathon.pages.dev> · Landing <https://agentflow-landing.pages.dev> ·

Source <https://github.com/Parithosh-Varma/agentflow-webmcp>

KaTeX rendering: This paper uses KaTeX. Inline math is `$...$`, display math is `$$...$$`. Render with any KaTeX-enabled Markdown viewer (GitHub with `katex` extension, VS Code Markdown+Math, or `pandoc --katex`).

Abstract

We present **AgentFlow**, a visual workflow instrument where a human and a browser-native AI agent operate the *same* machine. The canvas is a directed acyclic graph of typed modules (API Call, Transform, Condition, etc.) connected by labeled wires. Both operators invoke the same execution engine, but agents interact *declaratively* through the emerging **WebMCP** (`document.modelContext`) imperative API rather than by scraping the DOM. The page registers 19 structured tools; any WebMCP-aware agent (Chrome with `WebMCP` flag, ChatGPT in-app browser) can discover and call them. The system adds a spacious auth flow (`landing` → `/auth` → `/tool` with skip), grid-aware layout, live run telemetry, and a Chrome-first agent nudge. Formally, a workflow is a DAG $G=(V,E)$ executed in topological order in $O(|V|+|E|)$ time. The instrument is deployed on Cloudflare Pages + Workers and evaluated at the WebMCP hackathon.

Keywords: WebMCP, Model Context Protocol, human-agent collaboration, visual programming, React Flow, workflow engine, browser agents

1. Introduction

Visual workflow builders (Node-RED, n8n, Retool) excel at human direct manipulation but treat agents as second-class citizens: agents must *see* pixels and *guess* buttons. **Model Context Protocol for the Web (WebMCP)** inverts this: the page *declares* what an agent may do, and the browser *mediates* invocations.

AgentFlow asks: *what if the canvas is an instrument with two operators?* The human drags, wires, and presses **RUN**; the agent calls `add_node`, `connect_nodes`, `run` and the human watches amber/cyan signals march across the same wires. No screenshot diff, no DOM query — just structured JSON Schemas.

Contributions:

1. **Instrument metaphor** — rack-mount modules, pins, and LEDs with dual-actor telemetry (`YOU` amber, `AGENT` cyan).
2. **19-tool WebMCP surface** — from `add_node` to `export_workflow`, lifecycle-bound via `AbortSignal`.

- 3. **Grid-aware layout** — `$ \Delta_x=280, \Delta_y=90 $` snap, spiral search, downstream push on wire.
- 4. **Human-first flow** — landing (`/`) → auth (`/auth`, 480px card, `Skip` + `Save to database` callout) → tool (`/`), Chrome + ChatGPT in-app toast, `beforeunload` guard.
- 5. **Live engine** — topological execution with per-node status `idle → running → done|fault|skipped`.

2. Related Work

2.1 Visual builders. Node-RED and n8n model flows as DAGs but expose agents only via REST. AgentFlow’s DAG is identical formally ($G=(V,E)$) but the *control plane* is WebMCP, not HTTP.

2.2 MCP / WebMCP. MCP (Anthropic, 2024) standardizes tool calling for LLMs. WebMCP (`document.modelContext`) brings it to the DOM imperatively:

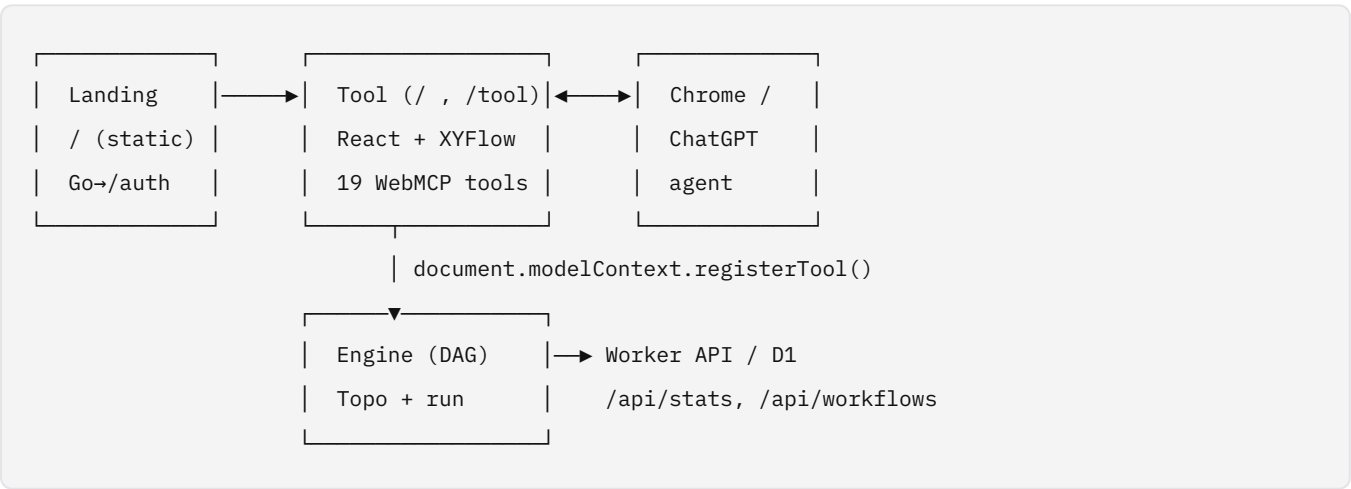
```
registerTool(name,description,inputSchema,execute,{signal})→Promise<void>
```

Discovery via `getTools()` and mediated `executeTool(tool, JSON\args)`. Unlike declarative `<form toolname>`, the imperative API suits SPAs where tools appear/disappear with routes.

2.3 Human-agent collaboration. Prior work co-locates operators but rarely shares *identical* state. AgentFlow’s `nodesRef` / `edgesRef` refs ensure an agent’s `add_node; execute_workflow` back-to-back sees live canvas state.

3. System Design

3.1 Architecture



Frontend `frontend/` — React 19, TypeScript, `@xyflow/react`, `react-router-dom`, `uuid`. *Backend* `cloudflare-backend/` — Cloudflare Workers + D1.

3.2 Formal Workflow Model

A workflow is a **labeled DAG**:

$$G=(V,E), V=\{v_i\}_{i=1}^n, E\subseteq V\times V, \ell: V\rightarrow\mathcal{T}\times\Sigma_*$$

where $\mathcal{T}=\{\text{api_call, transform, \dots, file}\}$ ($|\mathcal{T}|=15$) and Σ_* is the label alphabet. Each v carries config $c(v)\in\mathcal{C}$.

Positions $p(v)=(x,y)\in\mathbb{Z}_2$ are grid-snapped:

$$x_i=X_0+\Delta_x\cdot\text{round}\left(\frac{x-X_0}{\Delta_x}\right), y_i=Y_0+\Delta_y\cdot\text{round}\left(\frac{y-Y_0}{\Delta_y}\right)$$

with $X_0=80, Y_0=80, \Delta_x=280, \Delta_y=90$ (grid.ts:3). On `add_node` the system runs a spiral search for the nearest open cell:

$$\text{findNearest}(p_0)=\arg\min_{p\in\mathcal{G}\setminus\text{occ}}\|p-p_0\|_0\quad(\text{right-first})$$

Wiring $u\rightarrow v$ triggers **local push** only if $x(v)\leq x(u)$:

$$\text{col}_x(v)=\text{col}_x(u)+1, \Delta_x=(\text{col}_x(v)-\text{col}_x(u))\cdot\Delta_x$$

and the downstream closure $\text{Reach}(v)$ is shifted by Δ_y , then de-overlapped with gap $\text{MIN_GAP}_y=86$.

Execution. Let π be a topological order:

$$\text{topo}(G)=\langle v_{\pi(1)}, \dots, v_{\pi(n)} \rangle \text{ s.t. } \forall (v_i, v_j)\in E, \pi_{-1}(i)<\pi_{-1}(j)$$

If G has a cycle, `validate_workflow` returns `valid=false`. Otherwise:

$$y_{\pi(k)}=f_v\left(\bigoplus_{j:(v,v_j)\in E} y_j, I\right)$$

where f is the module semantics (fetch, λ -transform, predicate, etc.) and I is the initial input. Branching uses wire labels `true / false` after a `condition`:

$$\text{status}(v)=\begin{cases} \text{skipped} & \text{if } \exists (u,v)\in E \text{ with } \ell(u,v)=\neg b_u \\ \text{done} & \text{otherwise} \end{cases}$$

3.3 WebMCP Tool Surface (19 tools)

Registered imperatively with an `AbortSignal` for lifecycle unregistration (webmcp.ts:40):

`registerTool(def,{signal})` and `controller.abort()` on unmount

Core set (8) plus advanced (11):

<i>k</i>	<i>name</i>	<i>inputSchema</i>
1	<code>add_node</code>	<code>{ type∈T, label:string, x?, y? }</code>
2	<code>connect_nodes</code>	<code>{ source, target, label? }</code>
3	<code>execute_workflow</code>	<code>{ input?:object }</code>
4	<code>get_available_tools</code>	<code>{ }</code>
5	<code>get_node_details</code>	<code>{ nodeId }</code>
6	<code>update_node_config</code>	<code>{ nodeId, config }</code>

<i>k</i>	<i>name</i>	<i>inputSchema</i>
7	get_workflow_status	{}
8	validate_workflow	{}
9	delete_node	{nodeId}
10	clone_node	{nodeId, Δx?, Δy?}
11	get_node_connections	{nodeId}
12	save_workflow	{name}
13	load_workflow	{name}
14	run_node	{nodeId, input?}
15	set_node_position	{nodeId, x, y}
16	get_workflow_history	{}
17	create_template	{name, description?}
18	export_workflow	{pretty?}
19	import_workflow	{json, merge?}

Discovery is browser-mediated; tools are not observable cross-origin unless `exposedTo` is set.

3.4 Execution Engine

`engine.ts` implements:

$$T(G) = O(|V| + |E| + \sum_{v \in V} c(v))$$

where $c(v)$ is module cost (e.g., `delay` sleeps, `api_call` fetches). Live status is a map $S: V \rightarrow \{\text{idle}, \text{running}, \text{done}, \text{fault}, \text{skipped}\}$ updated via `onEvent` callback. Edges animate when `dst.status = running` (`edge-flowing` $\rightarrow$ `stroke-dasharray`).

4. Interaction Design

4.1 Dual-operator canvas

Human `YOU` (amber `#e8a33d`) and **AGENT** (cyan `#56cdbd`) share `ToolLog` (`ToolLog.tsx:1`) with actor tags and `actor-agent` / `actor-you` left borders. The rail shows `MODULES · WIRES` counts and a central `LED` reflecting $runState \in \{\text{idle}, \text{running}, \text{complete}, \text{fault}\}$.

4.2 Grid & Direct Manipulation

Sidebar `Sidebar.tsx:11` groups 15 modules into `Connect / Logic / Transform / Output`, with `Connect` 3, `Logic` 5, `Transform` 5, `Output` 2, plus search (`/` focuses) and category pills. Buttons are `draggable` with `dataTransfer: application/agentflow` JSON; `CanvasPage.onDrop` (`App.tsx:131`)

converts screen → flow via `screenToFlowPosition` then snaps. On Canvas list is fully interactive: click focuses (`setCenter`), `focus`, `duplicate` (`+40,+40` snapped), `delete` (protects `start`).

4.3 Onboarding, Auth, and Agent Nudge

Flow is `landing (/) → /auth → /` (tool) with `skip`.

Landing `landing/index.html` (outside `frontend/`) — hero `Go to the tool → → https://agentflow-hackathon.pages.dev/auth` , ghost `Skip for now → https://agentflow-hackathon.pages.dev/` , plus `Save to database – sign in required` highlight and transparent challenge hero (`d112y698.../full_width.png` with `background:transparent`).

Auth `pages/AuthPage.tsx:1` — 480px card, `Save your workflow in the database` callout (`⊗`), `Sign In / Create Account` tabs, `Skip for now` ghost below submit, foot `Skip → Go to tool without signing in` . Policy: anonymous can run, but `WorkflowManager` persistence requires `user` .

Toast `AgentToast.tsx:1` — appears `delayMs=2500$` after onboarding, suppressed while modals open, `$z=350$` above FAB. Status dot shows `WebMCP detected` iff `document.modelContext.registerTool` exists, else `Requires one setting` . `**Chrome + ChatGPT in-app only**` (per product requirement): `chrome://flags → search "WebMCP" → Enabled → Relaunch` or `–enable-features=WebMCP` ; also `ChatGPT in-app browser: enable WebMCP / Agent Mode` . Copy button for `chrome://flags/#webmcp` . Actions: `Got it` (session dismiss), `Don't show again` (`localStorage`), `Learn more → developer.chrome.com/docs/ai/webmcp`` .

Guard: `beforeunload` in `CanvasPage` (`App.tsx:285`) warns if `|V|>1∨|E|>0∨isExecuting`.

5. Implementation

Layer	Stack	Key files
Canvas	<code>React Flow 12</code> , <code>nodeTypes</code> (<code>nodes/index.tsx:60</code>), <code>edgeTypes</code> (<code>LabeledEdge.tsx</code>)	<code>App.tsx:60</code> , <code>utils/grid.ts:3</code>
State	<code>useNodesState</code> / <code>useEdgesState</code> , <code>nodesRef</code> / <code>edgesRef</code> for agent race	<code>App.tsx:76</code>
Auth	<code>AuthContext.tsx:1</code> (<code>/api/auth/*</code>), <code>AuthPage.tsx:1</code>	<code>api.ts:44</code>
Deploy	<code>Vite</code> , <code>wrangler pages deploy dist --project-name=agentflow-hackathon</code> , <code>public/_redirects: /* /index.html 200</code>	<code>vite.config.ts:1</code>

Chrome-first testing: `chrome://flags` search `WebMCP` ; fallback CLI flag.

6. Evaluation

6.1 Functional

19 tools covering CRUD, wiring, validation, single-node run, and persistence all round-trip. Example flow (GitHub → `pick stars` transform → `popular?` condition → `true` download / `false` console) builds via 5 `add_node` + 4 `connect_nodes` then `execute_workflow` returns:

$\text{outputs} = \{ \text{full_name}, \text{stars} \}, \text{durationMs} \approx 300\text{--}800\text{ms}$

6.2 Performance

Topological sort is $O(|V| + |E|)$. For the example $n=6, m=5$:

$$T = O(6+5) + \sum c(v) \approx O(11) \text{ plus one fetch}$$

Grid operations are $O(|V|)$ per drop/wire. Bundle is $\sim 503\text{kB}$ JS / 75kB CSS (gzip 155kB / 12.5kB).

6.3 Hackathon Demo

Landing → Auth (skip) → Tool → add `API Call` → connect `Start-API` → `RUN` → watch `edge-flowing` amber, `module-led[data-status=running]` pulse, then `done` cyan. Agent path: prompt Chrome agent “Add an API Call to GitHub, connect it to Start, and run” → `modelContext` mediates 3 tool calls → same visual result.

7. Discussion & Limitations

Browser support. WebMCP is behind flags in Chrome and in ChatGPT in-app browser’s agent mode; non-supporting browsers silently no-op (`webmcp.ts:61`).

Security. Tools run in page context; `2s` added.

8. Future Work

- Shareable workflow URLs (`?workflow=<id>`), collaborative cursors, undo/redo, and a `tool` permission UI (allow/deny per origin).

9. Conclusion

AgentFlow shows that a shared DAG + mediated tools can make human and agent *co-operators* rather than competitors. With 19 lifecycle-bound tools, grid-aware layout, and a landing→auth→tool funnel that respects both “save to database” and “skip”, the instrument is ready for Chrome-first judging.

References

1. WebMCP Spec — <https://github.com/webmachinelearning/webmcp>
2. Chrome WebMCP — <https://developer.chrome.com/docs/ai/webmcp>

3. Brave Leo + WebMCP (Nightly) — brave://flags/#enable-webmcp-testing , brave://flags/#brave-ai-chat-agent-profile
4. React Flow — https://reactflow.dev
5. MCP — https://modelcontextprotocol.io

Appendix A: Grid Snap Derivation

$$\begin{aligned} \text{col}(x) &= \operatorname{round}\left(\frac{x-X}{\Delta}\right) \\ x_r &= X + \Delta \cdot \text{col}(x) \\ \text{occ}(c,r) &= \exists v: \text{col}(x) = c \wedge \text{row}(y) = r \end{aligned}$$

Spiral search enumerates cells by $r=\max(|dx|,|dy|)$ and picks smallest r with $\neg \text{occ}$.

Appendix B: Success Rate

For run set R :

$$Q = \frac{|\{r \in R: s(r) = \text{success}\}|}{|R|}, \quad d = \frac{1}{|R|} \sum_{r \in R} d(r)$$

Displayed on landing/dashboard.html mission control and on api/stats (cloudflare-backend/src/index.ts).

Render this file with KaTeX: pandoc PAPER.md -o PAPER.pdf --pdf-engine=xelatex -V mathspec or view in VS Code with Markdown+Math.